1 Instruction to the Framework

1.1 Overview of the Framework's Architecture

This part provides an overview of the organization and layout of the files within the system. It explains how files are structured, including directory hierarchies.

- Root Directory
  - SRC
    - **main1.cpp** example usage1
    - **main2.cpp** example usage2
    - **main3.cpp** example usage3
    - **main4.cpp** example usage4
    - **main5.cpp** example usage5
    - framwork
      - **graphnode.h** graph node definition header file
      - **graphnode.cc** graph node implementation
      - **niobase.h** NIO data, a struct data that can be shared between different graph nodes
      - **register.h** register each graph node
      - **register.cc** register each graph node(implementation)
      - **registerdata.h** register NIO data, header file
      - **registerdata.cc** register NIO data, implementation
      - **util.h** some utility function, header file
      - **util.cc** some utility function, implementation
  - DATA
    - **a.out** exe to generate input data
    - **data.csv** input data
    - **main.cpp** source code to generate input
    - **output1_from_EXE4.csv** output1 from EXE4
    - **output_from_EXE4.csv** output from EXE4
    - **output_from_EXE5.csv** output from EXE5
    - **output_from_EXE2.csv** output from EXE2
    - **output_from_EXE3_AAPL.csv** output from EXE3_AAPL
    - **output_from_EXE3_GOOG.csv** output from EXE3_GOOG
    - **output_from_EXE3_JPM.csv** output from EXE3_JPM
  - **CMakeLists.txt** CMake configuration file
  - **README.txt** readme file

Next, let's introduce several important files and how the framework operates.

1.1.1 graphnode.h

GraphNodeBase is a virtual base class, and **all nodes on the computation graph inherit from this base class**. It has two important virtual functions: one is Initialize, used to initialize each node and register the data used; the other is LoadData, where users need to implement the specific computation logic.

```
/*
 * @brief this is an abstract class, users need to inherit from it to create their own computational logic
 *
 */
class GraphNodeBase
{
  public:
    class DataRegister *dr;//pointer to data register
    std::string tag;//the name of each node
    int mid;//module id
    GraphNodeBase() {}

    /*
     * @brief this is a virtual function, user need to use it to initialize each node
     * @param para is string, we can pass some parameter or ""
     */
    virtual void Initialize(const string &para) = 0;

    /*
     * @brief this is a virtual function, user need to implement the calculation logic here
     * @param para is string, we can pass some parameter or ""
     */
    virtual void LoadData() = 0;

    /*
     * @brief this function help to register graphnode's local variable, so that it can be used by other nodes
     * @param niodata pointer to some data
     * @param name can be used as a key, we can used this name as key(so that from other nodes we can used this key to find this node's data)
     */
    void AddDailyData(struct NIO_BASE *niodata, const string &name);

    /*
     * @brief each node keep's the pointer to a common DataRegister
     *
     */
    void SetDataRegister(class DataRegister * drptr) {dr = drptr;}
};
```

Figure 1: GraphNodeBase

1.1.2 niobase.h

NIO_BASE is **an abstract class related to data**. Its purpose is to allow each node to define different types of data inherited from the NIO_BASE class. Each graph node can access data from other nodes through a unified interface.

```
/*
 * @brief NIO_BASE is an abstract class
 */

struct NIO_BASE
{
  virtual void Clear() = 0;
};


/*
 * @brief NIO_MATRIX<T> can be used to define some data which can be used in each graph node
 */
template<class T>
struct NIO_MATRIX:public NIO_BASE
{
  T ptr_;
  NIO_MATRIX() {}
  T* GetBasePtr() { return &ptr_; }
  const T* GetBasePtr() const { return &ptr_; }
  void Clear() { }
};
```

Figure 2: NIO_BASE

1.1.3 register.h

The Register class is used to **manage the registration of all nodes**. The register.h file provides a REGISTER macro, which facilitates the generation of specific node objects using class name strings. For example, if we implement a graph node class name GraphNode1, GetClassFromName("GraphNode1") will return a pointer to an object with type GraphNode1.

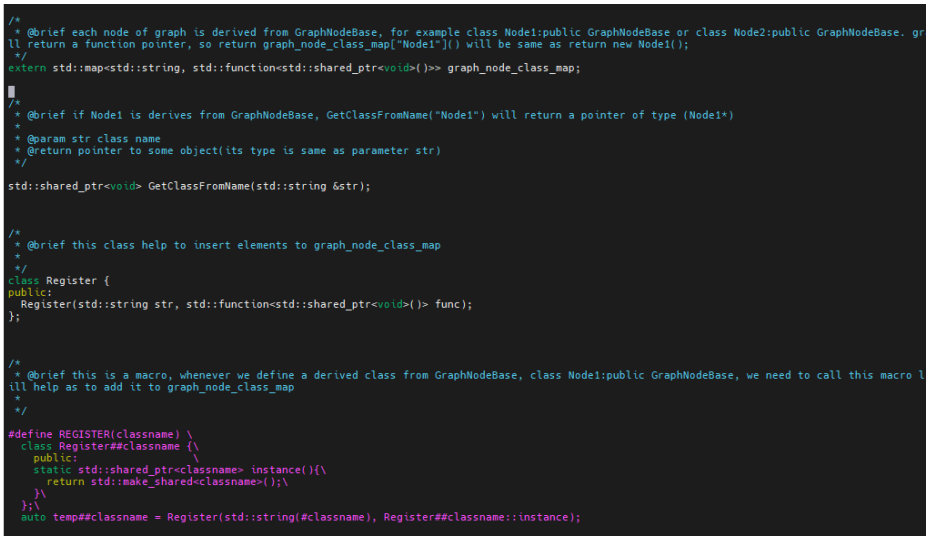

Figure 3: Register

1.1.4 registerdata.h

The RegisterData class is used to **manage the registration of the NIO_DATA class**. Only registered data can be shared across different graph nodes. We can obtain the registered data through the GetData method.

```cpp
#pragma once
#include <bits/stdc++.h>
using namespace std;



/*
 * @brief DataRegister help to keep the data used by each graph nodes
 *
 */

class DataRegister
{
public:
  std::map<std::string, int> nioname;
  std::map<int, int> niomidmp;
  std::vector<struct NIO_BASE*> nioptr;
  void RegisterNiodata(int mid, const std::string &name, struct NIO_BASE *niodata);


/*
 * @brief GetData help to get the reference of data used by some graph node
 * @param dataname is key to the find data reference
 * @return data reference of some data of type NIO_BASE
 */

  template<template<typename T> class A, typename B>
  A<B>& GetData(const std::string &dataname)
  {
    if (nioname.count(dataname) == 0) {
      std::cerr<<"can not find:"<<dataname<<std::endl;
    }
    if (nioname.count(dataname) <= 0) throw "nioname.count(dataname) <= 0";
    int off = nioname[dataname];
    if (nioptr[off] == NULL) throw "nioptr[off] == NULL";
    return *(dynamic_cast<A<B>*>(nioptr[off]));
  }
};
```

Figure 4: RegisterData

1.1.5 How Does the Framework Work

What the user should do:

- User needs to **implement graph nodes**(inherit from GraphNodeBase defined in graphnode.h), register.h will help to create the node object.
- If some node has data to be shared by other nodes, we need to **store it in structure NIO_BASE**(noibase.h), and register it with class RegisterData(registerdata.h)
- The computation **graph should be a DAG**, and we can use toposort to get the correct update order.
- When we need to update the graph, **update each node in topological order**.

What the framework do for us:

- Use a vector of pointer of GraphNodeBase to manage all the user defined nodes.

- Use a DataRegister to manage the data that shared between graph nodes.
- Update each node in topological order.

1.2 Introduction to Testing Data

We generate the testing data by using following code.

```cpp
#include <bits/stdc++.h>
#include <fstream>
using namespace std;



int main(void)
{

  int N = 100000;
  ofstream out("data.csv");
  vector<string> IIs = {"AAPL", "GOOG", "JPM"};
  vector<string> header = {"GOOD header", "BAD header", "NEUTRAL header"};
  vector<string> body = {"GOOD body", "BAD body", "NEUTRAL body"};

  int64_t start_unix_timestamp = 1744246595;

  for (int i = 0;i < N;++i) {
    string ii = IIs[rand() % IIs.size()];

    int prc = 100 + rand() % 10;//price is in range [100, 109] for each stock
    int qty = 1 + rand() % 1000;//qty is in range [1, 1000] for each stock

    string &local_header = header[rand() % header.size()];
    string &local_body = body[rand() % body.size()];
    start_unix_timestamp = start_unix_timestamp + 1 + rand() % 10;
    out<<start_unix_timestamp<<","<<ii<<","<<prc<<","<<qty<<","<<local_header<<","<<local_body<<endl;
  }

 return 0;
}
```

Figure 5: code to generate testing data

Followings are **some assumptions** we make about the data.

- There are 6 columns in total, timestamp, ticker, price, volume, headline, body of news.
- To make it simple, we use the unix timestamp.
- There are only 3 type of headlines: ”GOOD header”, ”BAD header”, ”NEUTRAL header”.
- There are only 3 type of body of news: ”GOOD body”, ”BAD body”, ”NEUTRAL body”.
- There are only 3 type of instruments: ”AAPL”, ”GOOG”, ”JPM”.
- To simplify, we randomly create a new record with random ticker.
- To simplify, price and volume are some random variable in certain range.

1.3 How to Compile and Run The Code

To install and build the project, follow these steps:

1.3.1 Install cmake and g++

Make sure your machine has cmake and g++

1.3.2 Navigate to the project directory

cd [project-name]

1.3.3 Create a build directory

mkdir build; cd build

if build exists, suggest remove it first.

1.3.4 Run CMake to configure the project

cmake ..

1.3.5 Build the project

make (or make -j20)

If everything is correct, we can successfully compile the project.

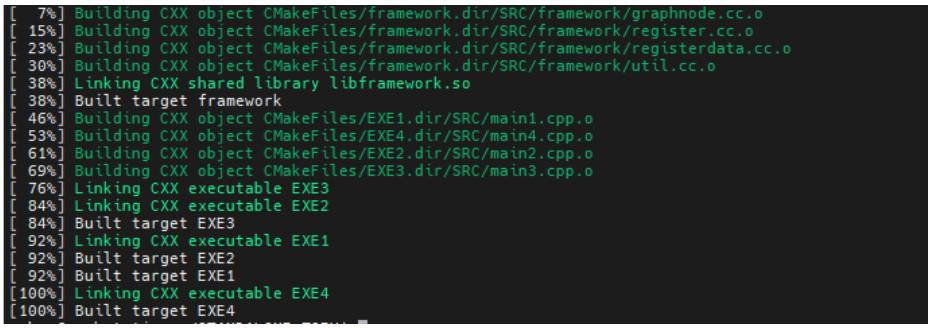

Figure 6: output of compiling

1.3.6 Run different examples

After building the project, you can run the executable from the build directory:

there are 5 different examples:

- ./build/EXE1 (define some graph nodes, calculating some PV features when good news or bad news happens, only for AAPL)

- ./build/EXE2 (define some graph nodes, calculating some PV features when good news or bad news happens, and save features for AAPL)

- ./build/EXE3 (define some graph nodes, calculating some PV features when good news or bad news happens, and save features for more than one instruments)

- ./build/EXE4 (this example is a more complex version, adding a graph node showing how to calculate a groupby algo in a stream way, and adding a node keep the feature when an hourly event happens)

- ./build/EXE5 (not related to event processing, showing other usage)

In next section, will explain each of these four examples.

2 Examples Showing How to Use the Framework

Here, five examples are provided, each located in main1, main2, main3, and main4, main5, respectively. These examples demonstrate different usage scenarios or approaches of the framework. They are independent code segments with some connections between them.

- main1: Illustrates how to add computation nodes, establish topological relationships, and perform calculations.
- main2: Builds upon main1 by introducing a new node that saves features.
- main3: Shows how to handle multiple stocks simultaneously.
- main4: Further extends main2 by adding more computation nodes and demonstrating the implementation of groupby logic in a streaming context.
- main5: Shows how to apply operations on some dataframe, which is useful for data mining.

Each example focuses on a specific aspect of the framework, providing a comprehensive understanding of its capabilities and flexibility.

2.1 main1.cpp

This section demonstrates how to **define the nodes of a graph, establish dependencies between nodes, generate the execution order of the nodes, and calculate all nodes in order** when there is information update. In this example, we assume that we are only processing the stock AAPL.

2.1.1 step 1 define graph node

Each graph node's calculation logic is different, so for each node we need to implement its logic. Following are some rules we need to obey.

- we can use NIO_DATA to define member variables, then it can be visited by other nodes
- if we define some NIO_DATA, we need to initialize it in Initialize function
- if we need to visit data from other graph nodes, we can use function GetData
- need to design the calculation carefully
- need to call the REGISTER at the end of definition

Here is an example of calculating the average price. Each time a new price data point is observed, we need to maintain the cumulative sum and the count of all samples observed so far. The average price is then obtained by dividing the cumulative sum by the count.

```
/* @brief StatMean keep the price mean
 * @param depend on price node
 * @return  mean price
 */

class StatMean:public GraphNodeBase
{
  NIO_MATRIX<double> sig;
  double cum_sum;
  double cum_cnt;
public:
  void Initialize(const string &para)
  {
    cum_sum = 0;
    cum_cnt = 0;
    AddDailyData(&sig, tag);
  }
  void LoadData()
  {
    const NIO_MATRIX<double> &nio_ft = dr->GetData<NIO_MATRIX, double>("price");
    const double& prc = *(nio_ft.GetBasePtr());


    //each time we get a new price observation, we need to update cummulative sum and cnt
    cum_sum += prc;
    cum_cnt += 1.0;

    double& output = *(sig.GetBasePtr());

    //mean price always = cummulative sum / cummulative cnt
    output = (cum_cnt > 0)? cum_sum / cum_cnt:0.0;
    std::cout<<"FRAMEWORK(" + tag + "):"<<output<<std::endl;
  }
};

REGISTER(StatMean);
```

Figure 7: example of mean price

2.1.2 step 2 store node names and class names

We assume that we have completed the code for all the classes of the graph nodes. We need to record them using arrays, where their names (which can be arbitrarily assigned) and the specific class names are recorded in two separate arrays.

```
vector<string> node;
vector<string> node_class_name;
node.push_back("time")              ; node_class_name.push_back("TimeLoc")          ;
node.push_back("volume")            ; node_class_name.push_back("VolumeLoc")        ;
node.push_back("price")             ; node_class_name.push_back("PriceLoc")         ;
node.push_back("price_diff")        ; node_class_name.push_back("StatDiff")         ;
node.push_back("price_mean")        ; node_class_name.push_back("StatMean")         ;
node.push_back("vol_sum")           ; node_class_name.push_back("StatSum")          ;
node.push_back("headline")          ; node_class_name.push_back("HeadLine")         ;
node.push_back("body")              ; node_class_name.push_back("Body")             ;
node.push_back("any_event")         ; node_class_name.push_back("EventMerger")      ;
node.push_back("headline_negative") ; node_class_name.push_back("HeadLineNegative") ;
node.push_back("headline_positive") ; node_class_name.push_back("HeadLinePositive") ;
node.push_back("body_negative")     ; node_class_name.push_back("BodyNegative")     ;
node.push_back("body_positive")     ; node_class_name.push_back("BodyPositive")     ;
```

Figure 8: store node names

2.1.3 step 3 add dependencies

Next, we add dependencies between the nodes based on the required computation order. Each node may depend on several other nodes or may have no dependencies at all. It is important to note that this directed graph should not have any cycles; otherwise, the computation cannot be completed.

```
map<string, vector<string>> str_dependency;

str_dependency["price_diff"] = {"price"};
str_dependency["price_mean"] = {"price"};
str_dependency["vol_sum"] = {"volume"};
str_dependency["headline_negative"] = {"headline"};
str_dependency["headline_positive"] = {"headline"};
str_dependency["body_negative"] = {"body"};
str_dependency["body_positive"] = {"body"};
str_dependency["any_event"] = {"headline_negative", "headline_positive", "body_negative", "body_positive"};
```

Figure 9: add dependencies

Here is the structure of the generated graph. The 'time' node has no dependencies, while the 'any_event' node depends on four other nodes.

```
time

price-->price_diff
     \->price_mean

volume--->vol_sum

headline--->headline_negative ------------> any event
        \-->headline_positive ----------/
                                       /
body--->body_negative-----------------/
        \-->body_positive------------/
```

Figure 10: structure of the generated graph

2.1.4 step 4 create object for each graph node

Then, we need to create objects for each node in the graph. Additionally, we need a DataRegister class to manage the data that can be shared among the graph nodes.

- due to C++ polymorphism, we can use an array of GraphBase base class pointers to record all node objects.
- eche node has a unique name.
- each node keeps the pointer to DataRegister

```
std::vector<shared_ptr<GraphNodeBase>> dmgr_ptr_(node.size(), nullptr);

class DataRegister dr;

for (int i = 0;i < (int)node.size();++i) {
  dmgr_ptr_[i] = std::static_pointer_cast<GraphNodeBase>(GetClassFromName(node_class_name[i]));
  dmgr_ptr_[i]->mid = i;
  dmgr_ptr_[i]->tag = node[i];
  dmgr_ptr_[i]->SetDataRegister(&dr);
  cout<<node[i]<<endl;
}
```

Figure 11: create object for each graph node

2.1.5 step 5 call Initialize function for each node

For each node, calling the Initialize function once completes data registration and sets up the initial state.

```
for (size_t i = 0;i < dmgr_order.size();++i) {
  dmgr_ptr_[i]->Initialize("");
}
```

Figure 12: call Initialize function for each node

2.1.6 step 6 update the graph

For each piece of related data, the LoadData function is called according to the node invocation order. In this example, we focus solely on the calculation of features for a single stock and the definition of events. Subsequent examples will involve more complex scenarios.

```cpp
ifstream in("DATA/data.csv");
string ln;
//10. for each line of data
while (getline(in, ln)) {//for each of data
  std::vector<std::string> tk = split(ln, ',');
  //1744246601,GOOG,106,778,BAD header,NEUTRAL body
  string local_ii = (tk[1]);
  if (local_ii != "AAPL") continue;

  //11. extract timestamp,price,qty,headline,body information for AAPL
  GLOBAL::global_timestamp = stoll(tk[0]);
  GLOBAL::global_price = atof(tk[2].c_str());
  GLOBAL::global_qty = atof(tk[3].c_str());
  GLOBAL::global_headline = std::move(tk[4]);
  GLOBAL::global_body = std::move(tk[5]);
  check_price_sum += GLOBAL::global_price;
  check_price_cnt += 1.0;

  //12. update nodes in correct order
  for (size_t i = 0;i < dmgr_order.size();++i) {
    int local_mid = dmgr_order[i];
    dmgr_ptr_[local_mid]->LoadData();
  }
  std::cout<<"DO NOT USE FRAMEWORK MEANPRC="<<check_price_sum / check_price_cnt<<std::endl;
}
```

Figure 13: update the graph when we recieve a new record

2.2 main2.cpp

The second example demonstrates how to extend the first example. For instance, if you want to **record the current time and some features when an event occurs and save the information into a specific file**, you can follow these steps to achieve it.

2.2.1 design a new graph node class

Based on main1, the goal of main2 can be achieved by making some extensions.

First, design a new class, FeatureDumper, to record specific information (such as timestamp, price_diff, price_mean, and vol_sum) when the event flag is set to 1.

```cpp
/* @brief if any event happens, we dump the timestamp and some feature
 */
class FeatureDumper:public GraphNodeBase
{
  ofstream out;
public:
  void Initialize(const string &para)
  {
    out.open("DATA/output_from_EXE2.csv", std::ios_base::out);
    assert(!out.fail());
  }
  void LoadData()
  {

    const NIO_MATRIX<int64_t> &nio_ft0 = dr->GetData<NIO_MATRIX, int64_t>("time");
    const NIO_MATRIX<int> &nio_ft1 = dr->GetData<NIO_MATRIX, int>("any_event");
    const NIO_MATRIX<double> &nio_ft2 = dr->GetData<NIO_MATRIX, double>("price_diff");
    const NIO_MATRIX<double> &nio_ft3 = dr->GetData<NIO_MATRIX, double>("price_mean");
    const NIO_MATRIX<double> &nio_ft4 = dr->GetData<NIO_MATRIX, double>("vol_sum");

    const int64_t& v0 = *(nio_ft0.GetBasePtr());
    const int& v1 = *(nio_ft1.GetBasePtr());
    const double& v2 = *(nio_ft2.GetBasePtr());
    const double& v3 = *(nio_ft3.GetBasePtr());
    const double& v4 = *(nio_ft4.GetBasePtr());

    if (v1 == 1) {//some event happened
      out<<v0<<","<<v2<<","<<v3<<","<<v4<<endl;
    }
  }
};
REGISTER(FeatureDumper);
```

Figure 14: design a new class

2.2.2 add node information to graph

Add a node to the graph with name feature_dumper.

```cpp
vector<string> node;
vector<string> node_class_name;
node.push_back("time")             ; node_class_name.push_back("TimeLoc")          ;
node.push_back("volume")           ; node_class_name.push_back("VolumeLoc")        ;
node.push_back("price")            ; node_class_name.push_back("PriceLoc")         ;
node.push_back("price_diff")       ; node_class_name.push_back("StatDiff")         ;
node.push_back("price_mean")       ; node_class_name.push_back("StatMean")         ;
node.push_back("vol_sum")          ; node_class_name.push_back("StatSum")          ;
node.push_back("headline")         ; node_class_name.push_back("HeadLine")         ;
node.push_back("body")             ; node_class_name.push_back("Body")             ;
node.push_back("any_event")        ; node_class_name.push_back("EventMerger")      ;
node.push_back("headline_negative") ; node_class_name.push_back("HeadLineNegative") ;
node.push_back("headline_positive") ; node_class_name.push_back("HeadLinePositive") ;
node.push_back("body_negative")    ; node_class_name.push_back("BodyNegative")     ;
node.push_back("body_positive")    ; node_class_name.push_back("BodyPositive")     ;
node.push_back("feature_dumper")   ; node_class_name.push_back("FeatureDumper")    ;
```

Figure 15: add new node to graph

2.2.3 add new dependency

Add dependency relationships, as the new node depends on the time, price_diff, price_mean, vol_sum, and any_event nodes, requiring these dependencies to be incorporated.

```
map<string, vector<string>> str_dependency;
str_dependency["price_diff"] = {"price"};
str_dependency["price_mean"] = {"price"};
str_dependency["vol_sum"] = {"volume"};
str_dependency["headline_negative"] = {"headline"};
str_dependency["headline_positive"] = {"headline"};
str_dependency["body_negative"] = {"body"};
str_dependency["body_positive"] = {"body"};
str_dependency["any_event"] = {"headline_negative", "headline_positive", "body_negative", "body_positive"};
str_dependency["feature_dumper"] = {"time", "price_diff", "price_mean", "vol_sum", "any_event"};
```

Figure 16: add new dependency

The structure of the graph becomes more complex after the update.

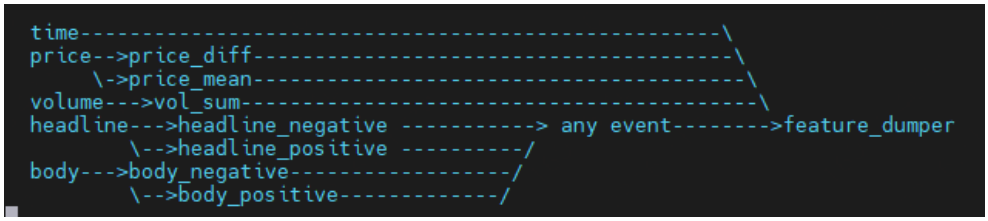

Figure 17: graph is more complex

2.2.4 update the graph and check the result

Run the program again, and it will record the current timestamp and volume-price features at the event occurrence, writing them into the file DATA/output_from_EXE2.csv. It has 4 columns in total, timestamp, price diff, mean price and cumulative volume. It only records the timestamp when any positive or negative event happens(skip neutral one).

```
1744246649,0,109,785
1744246658,-4,107,870
1744246669,0,106.333,1685
1744246678,-5,104.75,1773
1744246691,0,103.8,2450
1744246696,2,103.5,3103
1744246715,3,103.714,3813
1744246735,4,104.375,4084
```

Figure 18: run the program again and will get the output file

2.3 main3.cpp

The previous two examples handled the situation for a single stock. This example demonstrates how to **handle multiple stocks**. Since each stock is independent, we can add a computation graph for each stock, but to achieve this, some adjustments need to be made.

2.3.1 change some graph node class

For some input and ouput graph node, we need to keep the information of stock id, this stock id information can be used to the graph node to get the corret input and make the correct output. We need to modify the Initialize function a little bit.(following is the vimdiff between old and new)

Figure 19: change some the graph node class code

Figure 20: for the dumper we add ticker information to output file name

2.3.2 for each stock create a graph

Although we do not need to modify the topology of the graph, we need to **add the entire graph structure and an independent DataRegister** for each stock.

```
using GRAPHRUNNER = std::vector<shared_ptr<GraphNodeBase>>;
vector<GRAPHRUNNER> grunners(iis.size());//each stock has a runner
vector<class DataRegister> drs(iis.size());//each stock has a DataRegister


//7. for each stock, init the graph node independently
for (size_t z = 0;z < iis.size();++z) {
  auto &dmgr_ptr_ = grunners[z];
  //load ptr
  dmgr_ptr_ = std::vector<shared_ptr<GraphNodeBase>> (node.size(), nullptr);
  for (int i = 0;i < (int)node.size();++i) {
    dmgr_ptr_[i] = std::static_pointer_cast<GraphNodeBase>(GetClassFromName(node_class_name[i]));
    dmgr_ptr_[i]->mid = i;
    dmgr_ptr_[i]->tag = node[i];
    dmgr_ptr_[i]->SetDataRegister(&drs[z]);
    //cout<<node[i]<<endl;
  }
  //8. Before start the calculation logic, we need to call the Initialize function for each node.
  //It helps to initialize some states and register the data
  for (size_t i = 0;i < dmgr_order.size();++i) {
    dmgr_ptr_[i]->Initialize(iis[z]);//use ticker as a parameter
  }
}
```

Figure 21: for each stock create a graph

2.3.3 each stock one graph

For each new record, **find the corresponding graph for the current stock** and update the information.

```
ifstream in("DATA/data.csv");
string ln;
while (getline(in, ln)) {//for each of data
  std::vector<std::string> tk = split(ln, ',');
  //1744246601,GOOG,106,778,BAD header,NEUTRAL body

  string local_ii = (tk[1]);//

  int local_ii_id = GLOBAL::ii_to_id.at(local_ii);//get instrument's id

  GLOBAL::global_timestamp[local_ii_id] = stoll(tk[0]);
  GLOBAL::global_price[local_ii_id] = atof(tk[2].c_str());
  GLOBAL::global_qty[local_ii_id] = atof(tk[3].c_str());
  GLOBAL::global_headline[local_ii_id] = std::move(tk[4]);
  GLOBAL::global_body[local_ii_id] = std::move(tk[5]);

  //check_price_sum += GLOBAL::global_price;
  //check_price_cnt += 1.0;

  auto &dmgr_ptr_ = grunners[local_ii_id];
  for (size_t i = 0;i < dmgr_order.size();++i) {
    int local_mid = dmgr_order[i];
    dmgr_ptr_[local_mid]->LoadData();
  }

  //std::cout<<"DO NOT USE FRAMEWORK MEANPRC="<<check_price_sum / check_price_cnt<<s
}
```

Figure 22: update the graph

Three different files will be generated to record the volume-price features and event timestamps for each stock when an event occurs.

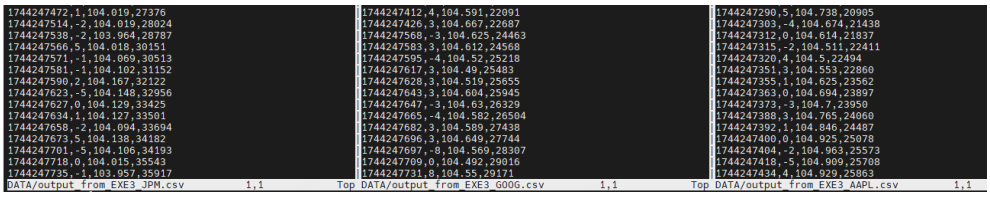

Figure 23: output for each stock

2.4 main4.cpp

In practice, we may need more complex computational logic and record samples corresponding to different events. This example demonstrates how to add a **groupby-related operator** and how to **simultaneously record multiple events** if needed.

2.4.1 a groupby_mean_std operator

As before, we need to add a groupby_mean_std-related operator(df[['price', 'volume']].groupby(by='price').mean().std()). In a streaming computation scenario, we need to record some states, such as **the sum and count of occurrences for each key**, as well as the **total count, total sum, and sum of squares for all keys**. For each new data point, we can update the relevant numerical values.

- For each key(price), we need to maintain its mean volume.
- We can use gbsum_cnt to keep each key(price)'s cumulative volume and count. So we can find the mean by deviding them.
- In order to find the std of all key's mean volume, we need to keep its mean(variable 'up' in the code), squre mean (variable 'up2' in the code)and count.
- for any new record, there will be two possible cases.
- (1) if the record's price **is not in gbsum_cnt**, we add (qty, 1) to gbsum_cnt, this new key's mean is v = gbsum_cnt / 1, so update up=up+v, up2 = up2 + v * v;
- (2) if the record's price **is already in gbsum_cnt**, we adjust gbsum_cnt[current price] by (qty, 1) , and change up and up2 in a different way, For details, see the code.
- Finally, the std can be found by using the information of the latest up, up2 and the size of gbsum_cnt.

```
class GroupByMeanStd:public GraphNodeBase
{
  NIO_MATRIX<double> sig;
  unordered_map<int, pair<double, double>> gbsum_cnt;
  vector<double> cache;
public:
  void Initialize(const string &para)
  {
    cache = vector<double>(3, 0);
    AddDailyData(&sig, tag);
  }
  void LoadData()
  {
    int64_t start_timestamp = TS();
    const NIO_MATRIX<double> &nio_ft0 = dr->GetData<NIO_MATRIX, double>("price");
    const NIO_MATRIX<double> &nio_ft1 = dr->GetData<NIO_MATRIX, double>("volume");
    const double& prc = *(nio_ft0.GetBasePtr());
    const double& qty = *(nio_ft1.GetBasePtr());

    int nid = static_cast<int>(prc);
    double v = qty;
    unordered_map<int, pair<double, double>>::iterator zz = gbsum_cnt.find(nid);

    double up = cache[1];//need to adjust it
    double up2 = cache[2];//need to adjust it

    if (zz != gbsum_cnt.end()) {
      double olds = (*zz).second.first;//gbsum[nid];
      double oldc = (*zz).second.second;//gbcnt[nid];
      (*zz).second.first = olds + v;
      (*zz).second.second = oldc + 1;
      double nv = (olds + v) / (oldc + 1);
      double ov = olds / oldc;
      up = up + nv - ov;
      up2 = up2 + nv * nv - ov * ov;
    } else {//a new one
      gbsum_cnt[nid] = make_pair(v, 1);
      up = up + v;
      up2 = up2 + v * v;
    }

    double down = gbsum_cnt.size();
    double mm = (down > 0)?up / down:0.0;
    double sm = (down > 0)?up2 / down:0.0;
    double ss = 0;//sigma
    if (down > 1) {
      double adj = (double) (down) / (double)(down - 1);
      ss = sqrt(sm - mm * mm) * sqrt(adj);
    }
    cache[0] = ss;
    cache[1] = up;
    cache[2] = up2;
    double& output = *(sig.GetBasePtr());
    output = cache[0];
    int64_t end_timestamp = TS();
    GLOBAL::macro_seconds_used_by_framework += end_timestamp - start_timestamp;

    std::cout<<"FRAMEWORK((groupbymeanstd)):"<<output<<std::endl;
  }
};

REGISTER(GroupByMeanStd);
```

Figure 24: a groupby_mean_std operator

We also need a node which reads in time stamp and detect the transition to a new hour. (This is an hourly event, 1 if the timestamp first time gets into a new hour, other case 0)

```
class HourEvent:public GraphNodeBase
{
  NIO_MATRIX<int> sig;//merge many VD to VVD
  int64_t oldti;
public:
  void Dependencies() { }
  void Initialize(const string &para)
  {
    AddDailyData(&sig, tag);
  }
  void LoadData()
  {

    const NIO_MATRIX<int64_t> &nio_ft0 = dr->GetData<NIO_MATRIX, int64_t>("time");
    const int64_t& newti = *(nio_ft0.GetBasePtr());

    int& output = *(sig.GetBasePtr());

    if (newti / 3600 != oldti / 3600) {
      output = 1;
    } else {
      output = 0;
    }
    oldti = newti;
  }
};
REGISTER(HourEvent);
```

Figure 25: for the dumper we add ticker information to output file name

2.4.2 structure of the graph

The final generated topology is more complex than before, and in this example, we also write groupby-related factors as features into the file.

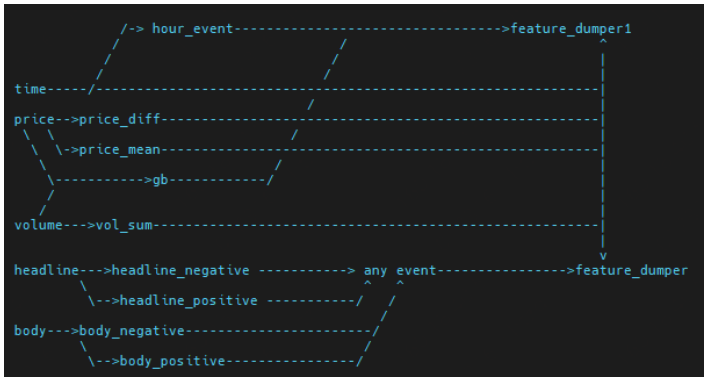

Figure 26: structure of the graph

In summary, there are **two types of events** in the graph:

- any_event(any news related event), if any positive news or negative news or positive body or negative body event happens, we set any_event to 1.
- hour_event, if enter an new hour, we set it to 1.

There are **four price volume related features**:

- price_diff:price diff
- price_mean:price mean
- vol_sum:sum of volume
- gb : df[['price', 'volume']].groupby(by='price').mean().std()

There are **two feature dumpers**:

- feature_dumper1:dump hour_event related feature
- feature_dumper:dump new event related feature

2.4.3 update the graph

At each update, we can directly compute to verify whether the framework provides the correct results. For each event, the code print two lines to the sceen, one is the feature score by using the framework, another one is the feature score we calculate without the framework. **If everything is correct, those two scores should be the same**.

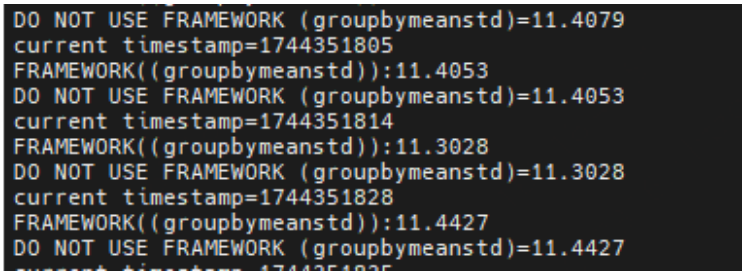

Figure 27: two methods to find groupbymeanstd

As expected, this program generates two output files. One is related to news event timestamps, and the other is related to hourly events (including groupby features, so an additional column is added).

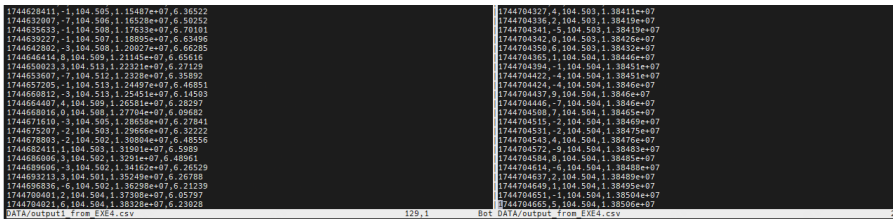

Figure 28: output files

Additionally, the program prints out the total time consumption of two calculation methods at the end, and it can be seen that the time consumption of stream processing is **significantly less** than that of direct calculation.

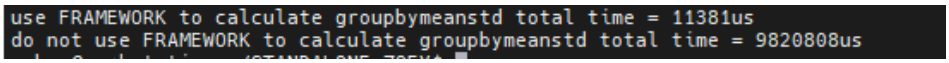

Figure 29: time comparison

2.5 main5.cpp

This part provides an example that is not closely related to event-driven architecture. In this example, we demonstrate **how to handle operations related to DataFrames** using this framework. We want to do following things: **based on one dataframe as input, try to extract four differnt features from it and save in csv**.Here, we present a new operator example based on DataFrames.(for details, please refer to source code)

```
class ColDemean:public GraphNodeBase
{
  NIO_MATRIX<vector<vector<double>>> sig;//store the demean result
public:
  void Initialize(const string &para)
  {
    AddDailyData(&sig, tag);
  }
  void LoadData()
  {
    const NIO_MATRIX<vector<vector<double>>> &nio_ft = dr->GetData<NIO_MATRIX, vector<vector<double>>>("randommat");
    const vector<vector<double>>& df = *(nio_ft.GetBasePtr());
    vector<vector<double>>& output = *(sig.GetBasePtr());
    int n = df.size();
    int m = df[0].size();
    output = vector<vector<double>>(n, vector<double>(m, 0));//matrix of size n * m

    for (int i = 0;i < m;++i) {
      double s = 0;
      for (int j = 0;j < n;++j) {
        s += df[j][i];
      }
      s = s / m;//row mean
      for (int j = 0;j < n;++j) {
        output[j][i] = df[j][i] - s;
      }
    }
  }
};

REGISTER(ColDemean);
```

Figure 30: new node related to dataframe op

We create a different computation graph in this example. Based on one random matrix as input, we apply some opertions on it(to extract some features. Finally, we use a tocsv node to merge all features to a csv file.

```
randommat-->mat1-->mat1_squaresum------->tocsv
          |      \->mat1_abssum---------/
          |->mat2-->mat2_squaresum-----/
                 \->mat2_abssum-------/
```

Figure 31: computation graph

In this example, **we only need to update the graph one time** to get following csv as output. In practice we can use a larger graph to generate different types of features for data mining purpose.

```
,mat1_abssum,mat1_squaresum,mat2_abssum,mat2_squaresum
0,711.753,22909.6,1380.06,86315
1,717.059,22681.6,1687.95,115903
2,670.672,21031.2,1583.01,104191
3,849.115,29498.6,1532.56,107596
4,805.558,28661.8,1804,133291
5,752.597,24887,1501.83,100136
6,818.157,28058,1720.62,124894
7,764.759,25749.5,1604.79,111128
```

Figure 32: output of main5